Figure 1: DOM query and update

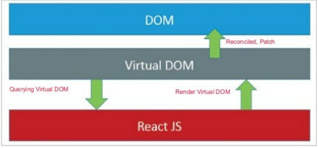

Figure 2: The virtual DOM of React

Figure 3: The React main page

The React approach is different. It only interacts with the virtual DOM, which is a JavaScript object as shown in Figure 2. When we use *getDOMNode*, we get the state of the DOM, and when we call the render function, React will update the virtual DOM which is nothing but a JavaScript object. Virtual DOM will push only the necessary changes to the real DOM, i.e., it reconciles with the real DOM.

Downloading the starter kit

React can be downloaded from *https://facebook.github.io/react/downloads.html*. As shown in Figure 3, click on 'Download React' and select 'Download the starter kit'. It has certain examples to demonstrate how React can be used. Extract the *Zip* file which will have two folders—*Build* and *Examples*. The *Build* folder contains all the required JavaScript files and the *Examples* folder has different examples of React.

Alternatively, we can use the script tag in our HTML file to link to React as shown in the following code:

```
<script src="https://fb.me/react-0.14.8.min.js"></script>
<script src="https://fb.me/react-dom-0.14.8.min.js"></script>
```

For this article, we will follow the latter approach.

```
<html>
    <head>
        <title>Hello World</title>
        <script src="https://fb.me/react-0.14.8.min.js"></script>
        <script src="https://fb.me/react-dom-0.14.8.min.js"></script>
    </head>
    <body>
        <script>
            React.render(React.createElement('div', null, 'Hello World'), document.body)
        </script>
    </body>
</html>
```

Figure 4: 'Hello World' example using React

Figure 5: Hello World sample output

```
<html>
    <head>
        <title>Hello World</title>
        <script src="https://fb.me/react-0.14.8.min.js"></script>
        <script src="https://fb.me/react-dom-0.14.8.min.js"></script>
        <script src="https://cdnjs.cloudflare.com/ajax/libs/babel-core/5.8.23/browser.min.js">
        </script>
    </head>
    <body>
        <script type="text/babel">
            var HelloWorld = React.createClass({
                render: function(){
                    return (<div>
                            <h1>Hello World</h1>
                            <h2>Welcome to React World</h2>
                        </div>);
```

Figure 6: HelloWorld using JSX

A sample application

Let us start with a simple application of displaying 'Hello World'. To display the output, we need to use the *React.render* function, which takes two arguments — what is to be printed and where. For printing 'Hello World', we will use *React.createElement* which is similar to a JavaScript element. It takes the tag name, *properties object*, and the variable number of the child arguments. We will create the *div* tag, which will display the 'Hello World' contents. Figure 4 shows the code of the *helloworld.html* file.

Open the HTML file in a browser; it will display 'Hello World' as shown in Figure 5.

Figure 7: Output of HelloWorld using JSX

```
    <head>
        <title>Hello World</title>
        <script src="https://fb.me/react-0.14.8.min.js"></script>
        <script src="https://fb.me/react-dom-0.14.8.min.js"></script>
        <script src="https://cdnjs.cloudflare.com/ajax/libs/babel-core/5.8.23/browser.min.js">
        </script>
    </head>
    <body>
        <script type="text/babel">
            var HelloWorld = React.createClass({
                render: function(){
                    return (<div>
                            <h1>Hello {this.props.uname}</h1>
                            <h2>Welcome to React World</h2>
                        </div>);
                }
            });

            React.render(<HelloWorld uname="Ashish"/>, document.body);
        </script>
    </body>
</html>
```

Figure 8: HelloWorld using Properties

Figure 9: Output of HelloWorld using Properties